STAT 578 Mini-Project II

Part 2 (Probe) and Part 3 (Connect)

Aarush Khirwadkar and *[teammate]*

Setup

CIFAR-10. ViT encoder with dim 192, depth 6, 3 heads and 4×4 patches, so $N = 64$. Light decoder with dim 96, depth 2. AdamW (lr 1.5×10^{-3}, weight decay 0.05), 10 warm-up epochs then cosine decay, batch 256. **Every run is 100 epochs with seed 0**, one run per configuration.

Linear probe. The encoder is frozen and the features standardised. A linear classifier is trained with AdamW for 40 epochs, and we report CIFAR-10 test accuracy.

- **Mean pooling is the headline number for every run**, since the assignment asks for one pooling choice used consistently. CLS pooling is shown alongside.
- The probe itself is not seeded: re-probing the same checkpoint moved accuracy by up to 0.2 points.
- Run-to-run noise from the pretraining seed is about 1–2 points.

All figures are regenerated from the saved `metrics.csv` files by `python make_figures.py`.

run	target	pattern	ratio	probe mean (%)	probe CLS (%)	final loss
`mae_r0.15`	pixel	random	0.15	57.74	50.29	0.1281
`mae_r0.50`	pixel	random	0.50	63.44	60.98	0.2338
`mae75`	pixel	random	0.75	68.19	60.68	0.3866
`mae_r0.90`	pixel	random	0.90	66.49	63.50	0.5736
`mae_block`	pixel	block	0.75	**68.52**	**65.99**	0.5565
`mae_grid`	pixel	grid	0.75	65.19	64.76	0.3394
`feat75`	feature (centered)	random	0.75	57.23	55.72	0.0651
`feat_nocenter`	feature (uncentered)	random	0.75	58.65	58.04	0.0845

Table 1: Linear-probe accuracy over all eight runs. `mae75` and `feat75` are the Part 1 reference runs; `mae75` is both the ratio-0.75 point and the random-pattern point.

1 Part 2(a): the masking distribution π

Q1. Probe accuracy and final loss vs. masking ratio

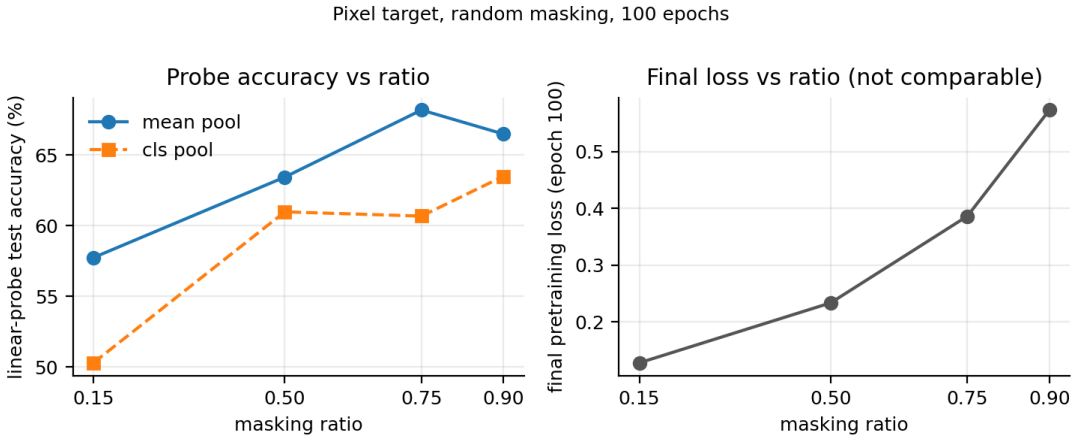


Figure 1: Pixel target, random masking, 100 epochs. Left: linear-probe accuracy. Right: final pretraining loss, which is not comparable across ratios.

The two panels have different shapes.

- **Loss** rises monotonically with the ratio: $0.128 \to 0.234 \to 0.387 \to 0.574$.
- **Mean-pool probe accuracy** is an inverted U: $57.7 \to 63.4 \to \mathbf{68.2} \to 66.5$, peaking at 0.75.
- **CLS probe accuracy** rises: $50.3 \to 61.0 \to 60.7 \to 63.5$.

By the loss, ratio 0.15 is the "best" run. By what the representation can do, it is the worst.

Q2. Why the loss is not comparable across ratios, and what changes about the prediction problem

The loss at each ratio has a different floor. The pixel loss averages, over hidden patches, the squared error against per-patch-standardised pixels:

$$\mathcal{L}_r(\theta) = \mathbb{E}_{X,\, S\sim\pi_r} \frac{1}{|S|} \sum_{t\in S} \tfrac{1}{48} \big\| q_\theta(X_{\bar{S}})_t - Y_t \big\|^2.$$

Its population minimiser is the conditional mean $\mathbb{E}[Y_t \mid X_{\bar{S}}]$. So the lowest achievable value is a Bayes risk that *depends on* r:

$$\mathcal{R}^\star(r) = \mathbb{E}_{S\sim\pi_r} \frac{1}{|S|} \sum_{t\in S} \tfrac{1}{48} \operatorname{tr} \mathbb{E}\big[\operatorname{Var}(Y_t \mid X_{\bar{S}})\big].$$

Raising r raises the floor. It shrinks the conditioning set: $54 \to 32 \to 16 \to 6$ visible patches out of 64. By the law of total variance, conditioning on less can only increase the expected conditional variance, so $\mathcal{R}^\star(r)$ grows with r. It also changes which patches are averaged over (Table 2).

So each ratio is a different estimation problem. A low loss at $r = 0.15$ means the problem is easy, not that the encoder is good. The loss compares each model against its own moving yardstick. The probe instead evaluates every encoder on the same downstream task, which is why it is the comparable quantity.

What the scale means. With `norm_pix`, every target patch has mean 0 and unbiased variance 1 over its 48 values. Predicting 0 therefore gives a loss of exactly $47/48 = 0.979$ (checked numerically in Q5). Dividing by 0.979 gives the fraction of per-patch variance left unexplained: 13% at $r = 0.15$, 24% at 0.50, 39% at 0.75 and 59% at 0.90.

pattern	ratio	visible patches	hidden with no visible 4-nbr	mean visible 4-nbrs	mean dist. to nearest visible	positions ever visible
random	0.15	54	0.002	3.00	1.00	1.0
random	0.50	32	0.087	1.78	1.09	1.0
random	0.75	16	0.357	0.89	1.43	1.0
random	0.90	6	0.700	0.33	2.30	1.0
block	0.75	16	0.626	0.41	2.14	1.0
grid	0.75	16	0.391	1.17	1.46	0.5

Table 2: Geometry of the masking distribution, computed from `masking.py` alone (4000 sampled masks, no data, no training). "4-nbr" means the four grid neighbours of a hidden patch; distances are grid L_1 distances.

What changes about the prediction problem.

- **At $r = 0.15$:** 99.8% of hidden patches have a visible 4-neighbour, with 3.0 visible neighbours on average. The task is *interpolation*: fill a hole from the pixels around it.
- **At $r = 0.90$:** 70% of hidden patches have *no* visible neighbour, and the nearest visible patch is 2.3 steps away on average. $\mathbb{E}[Y_t \mid X_{\bar{S}}]$ now depends on what the object is and where it extends. The task becomes *inference of global structure* from six patches.
- **In between, two things happen together.** The conditioning set gets too small for local continuation, and the conditional mean becomes a blur over many plausible completions. That blur is why reconstructions get blurry and the loss keeps rising.
- **Too high a ratio leaves too little to infer from.** This is a plausible reason that 0.90 falls 1.7 points below 0.75. That gap is close to seed noise, though.

Q3. Why a locally solvable task gives a weaker global representation, and the diagnostic that shows it

The argument. The objective only rewards getting $\mathbb{E}[Y_t \mid X_{\bar{S}}]$ right.

- **Low ratio.** That conditional mean is almost a function of the three visible neighbours of t: continue the edges and colours across the hole. The encoder reaches near-Bayes loss with features that encode *local, low-level appearance and position*. The identity of the object adds almost no predictive information once the neighbours are known. In the lecture's terms, the objective is *indifferent* between getting the ear's exact brown right and recognising that it is an ear. Since class information is not needed for the loss, it is not learned, and a linear probe reads exactly that information.

- **High ratio.** The conditional mean depends on the object's identity, pose and extent, so the encoder must represent them.

A mechanistic half. The CLS token is never reconstructed. It only gets gradient insofar as the decoder uses it, which happens through attention when the patch tokens are not enough. At low ratio the visible patch tokens are enough, so the CLS is left unused.

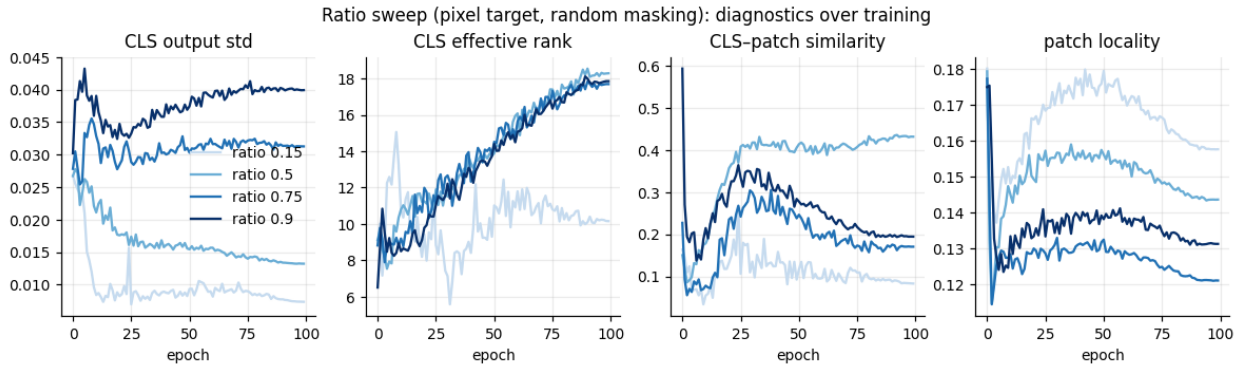


Figure 2: Ratio sweep (pixel target, random masking): CLS-level and patch-level diagnostics over training.

The diagnostic: CLS output std and effective rank over training (Figure 2).

- **At $r = 0.15$, `cls_std` collapses** from 0.027 at epoch 0 to 0.008 by epoch 10, and stays near 0.007. The CLS vector becomes nearly the same for every image.
- **Its effective rank peaks** at 15 (epoch 8), then decays to 10.
- **At every $r \geq 0.5$,** effective rank keeps rising to about 18, and `cls_std` ends 1.8–5.4× larger.
- **The ordering matches the CLS probe.** Final `cls_std` runs $0.0074 < 0.0132 < 0.0313 < 0.0399$. CLS-probe accuracy runs $50.3 < 61.0 \approx 60.7 < 63.5$.
- **This is shortcut-finding, not just weak features.** The 0.15 run *loses* global information while its loss keeps falling.
- **Patch locality does not separate the ratios cleanly** (0.121–0.158), so we do not rely on it.

Is this just comparing `mae_r0.90` with `mae_r0.15`? Essentially yes: the claim is about the sweep. Three refinements matter, though.

1. **Use all four points.** The $0.75 > 0.90$ reversal shows the relationship is not "higher is always better".
2. **Compare diagnostics and probe accuracy, not the loss.** The diagnostics measure the encoder's features on the same images for every run, so they *are* comparable across ratios. The loss is not.
3. **Look at trajectories, not only final values.** The collapse of the CLS at $r = 0.15$ by epoch 10 is stronger evidence than two endpoint numbers. The gap between 0.15 and the rest is 6–11 points, far beyond the 1–2 points of seed noise.

Q4. Masking pattern at ratio 0.75: block vs. grid

Prediction, from the masking geometry and the lecture's interpolation argument: block beats grid.

- **Block masking** hides 2×2 squares, so many hidden patches have no visible neighbour. That removes the interpolation shortcut, which is BEiT's reason for blockwise masking.
- **Grid masking** spreads the visible patches as evenly as possible, so interpolation stays available.
- **Expected order:** loss block > random > grid, and probe accuracy block > grid.

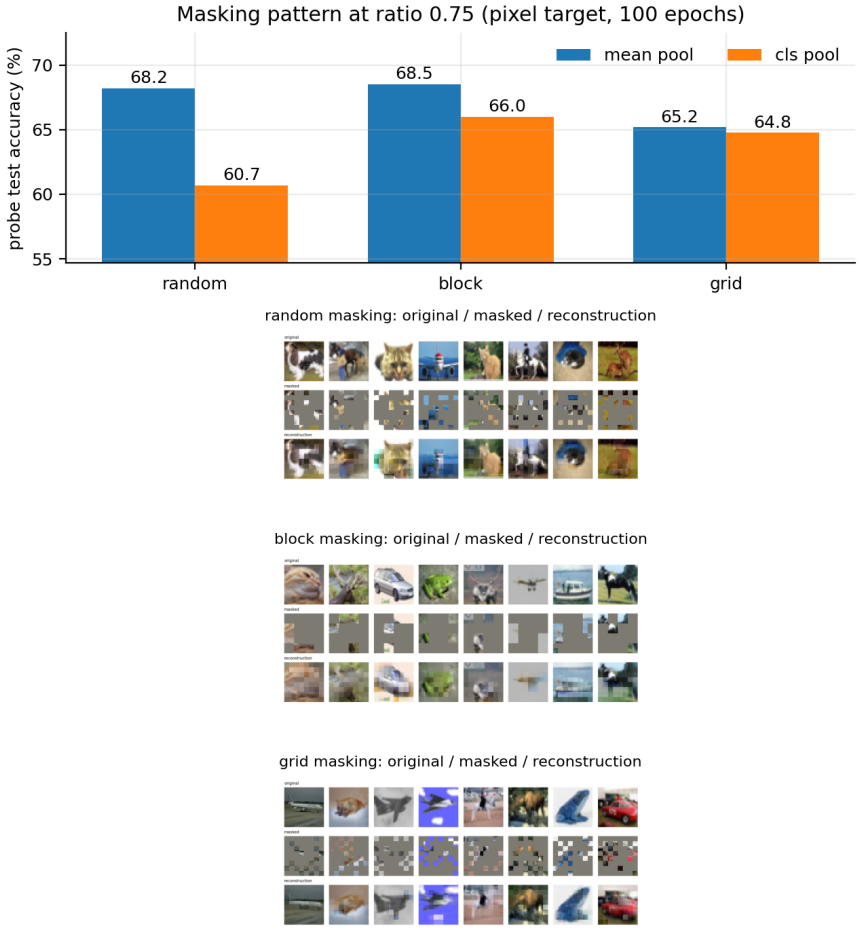


Figure 3: Masking pattern at ratio 0.75 (pixel target, 100 epochs). Top: probe accuracy. Below: original / masked / reconstruction grids for random, block and grid masking.

Result (seed 0, one run per pattern): block won, as predicted (Table 1, Figure 3).

- **Mean pool:** block 68.5% vs. grid 65.2%, a gap of 3.3 points.
- **CLS:** block 66.0% vs. grid 64.8%, a gap of 1.2 points.
- **Random** sits at 68.2% (mean pool).

Caveat: this is a single pretraining seed per pattern. The assignment puts run-to-run noise at 1–2 points. The CLS gap is inside that range and the mean-pool gap is only just outside it. We did not repeat the runs with a second seed, so "block beats grid" is suggestive, not established. The loss ordering and the masking geometry below are a separate line of evidence, and they point the same way.

Why block beats grid. The loss ordering confirms the mechanism: grid 0.339 < random 0.387 < block 0.557. That is exactly the order of **how many visible neighbours a hidden patch has on average**: grid 1.17, random 0.89, block 0.41 (Table 2).

- **This statistic of π separates block from grid at equal ratio**, and it needs no training.
- **The "no visible neighbour" fraction does not separate grid from random** (39% vs. 36%). Grid does not leave *more* hidden patches isolated. It packs the visible patches so that the hidden patches next to them get as many visible neighbours as possible, which is the best possible setting for interpolation.
- **The reconstruction grids show the same thing.** Grid reconstructions are the sharpest, and block reconstructions are blurry 2×2 squares.
- **The easier the local problem, the weaker the representation**, which is the Q3 argument again.

A second effect we did not predict, honestly diagnosed. The grid mask in `masking.py` is not random over positions:

- **How it works.** Scores are checkerboard parity $+ 0.5 \cdot U(0, 1)$, and the 16 lowest scores are kept. All 16 always land on the 32 *even-parity* cells ("positions ever visible" = 0.5 in Table 2).
- **The consequence.** During pretraining the encoder **never processes a patch at an odd-parity position**. At probe time it sees all 64 patches, half of them at positions it never saw as input.
- **Why this hurts mean pooling most.** This train/test shift should hurt mean-pooled patch features, which average over those tokens, more than the CLS. Consistent with that, grid's mean–CLS gap is 0.4 points, against 2.5 for block and 7.5 for random.
- **What we can conclude.** Grid's deficit is plausibly part "easier interpolation" and part "positional shift", and these runs cannot separate the two. A grid variant that randomises the checkerboard parity per image would.

Block vs. random (68.5 vs. 68.2 mean pool) is within noise. Hiding more local context than random masking already does at 0.75 bought little on the mean-pool probe. The CLS probe differs much more (66.0 vs. 60.7).

2 Part 2(b): the target Y

Q5. The constant solution

The loss. For the feature target, each hidden patch contributes

$$\ell_t = \|\hat{q}_t - \hat{y}_t\|^2 = 2 - 2\langle \hat{q}_t, \hat{y}_t \rangle \ \geq 0, \qquad \hat{q}_t = \frac{q_\theta(X_{\bar{S}})_t}{\|q_\theta(X_{\bar{S}})_t\|}, \quad \hat{y}_t = \frac{f_\xi(X)_t - \text{center}}{\|f_\xi(X)_t - \text{center}\|},$$

where f_ξ is the EMA teacher. Equality holds iff $\hat{q}_t = \hat{y}_t$.

The constant solution.

- The encoder outputs the same vector c for every patch of every image. For example, set the final LayerNorm's weight to 0 and its bias to c.
- The teacher is an EMA of the student's parameters, so it converges to the same constant map $f_\xi \equiv c$.
- The decoder outputs c for every hidden patch.

It minimises the feature loss. Without centering, $\hat{y}_t = \hat{q}_t = c/\|c\|$, so $\ell_t = 0$ for every image, patch and mask. Since $\mathcal{L} \geq 0$, this is a **global minimum**, and the gradient is zero too.

- **The starter code's centering does not change this.** Its running center converges to $c/2$ (shown in Q7), so $\hat{y} = \text{normalize}(c/2) = c/\|c\|$ again.
- **Checked numerically** (Table 3): the loss is 0 to floating-point precision ($\leq 10^{-7}$) with centering off *and* on.

model (one batch of 32 test images, ratio 0.75)	masked loss
feature target, constant encoder, centering off	1.2×10^{-7}
feature target, constant encoder, centering on (starter code)	0
pixel target, constant prediction 0	0.979
pixel target, trained `mae75`	0.485

Table 3: Numerical check of the constant solution.

Why the pixel loss has no such solution.

- **The target cannot move.** Y_t is a fixed function of the input (the standardised pixels of patch t), not of θ, so it cannot come to meet the prediction.
- **A constant prediction $q \equiv a$ pays the full variance.** Its loss is $\mathbb{E}\|Y_t - a\|^2/48 \geq \operatorname{tr}\operatorname{Cov}(Y_t)/48 = 47/48 \approx 0.979$, minimised at $a = \mathbb{E}Y_t = 0$.
- **The minimiser must depend on the input.** It is $\mathbb{E}[Y_t \mid X_{\bar{S}}]$, which varies because images differ.
- **The only way to lower the pixel loss is to carry information about the image.** In the feature loss both sides are functions of the parameters (θ and its average ξ). That makes "agree by both saying nothing" available.

Q6. The collapse panel: when the loss says success and when the diagnostics say something is wrong

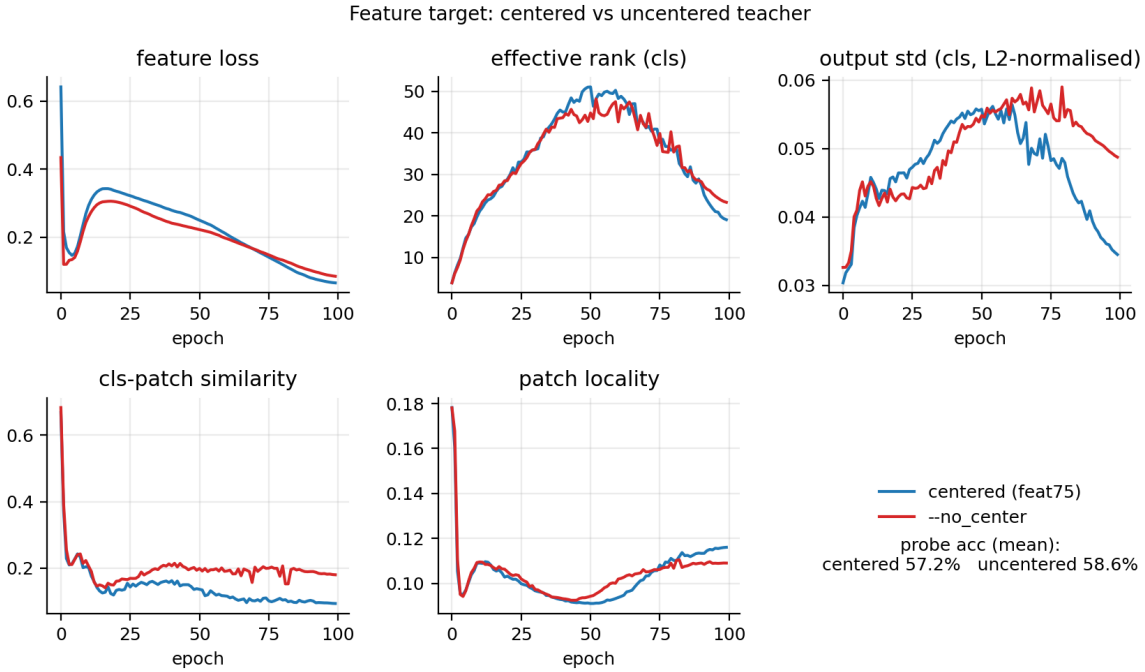


Figure 4: Feature target, centered (`feat75`) vs. uncentered (`feat_nocenter`): loss, effective rank, output std, CLS–patch similarity and patch locality on shared axes.

	centered (`feat75`)	uncentered (`feat_nocenter`)
loss minimum (epoch)	99	99
early loss low point (epoch)	4	1
loss / effective rank there	0.146 / 12.3	0.120 / 6.1
loss next this low (epoch)	74	84
effective-rank peak (epoch)	50	52
effective rank, peak → final	51.1 → 19.1 (−63%)	48.2 → 23.3 (−52%)
loss, rank peak → final	0.250 → 0.065 (−74%)	0.218 → 0.085 (−61%)
`cls_std` peak (epoch)	61	79
`cls_std`, peak → final	0.0564 → 0.0345 (−39%)	0.0590 → 0.0488 (−17%)
patch locality best (epoch)	50	44
locality, best → final	0.091 → 0.116 (+27%)	0.092 → 0.109 (+18%)

Table 4: Key epochs read off the collapse panel.

The loss alone would have declared success at epoch 99, the last epoch. In both runs the feature loss falls monotonically from epoch 17 (centered) / 19 (uncentered) and reaches its minimum at epoch 99, so the loss-based choice is "train to the end, keep the final checkpoint". There is also an earlier trap.

- **The early low point.** At epoch 1 (uncentered) and epoch 4 (centered) the loss had already fallen 70–80%.

- **The loss does not get that low again until epoch 84 (uncentered) / 74 (centered).** If you had stopped on that early plateau you would have declared success with a CLS effective rank of 6–12 out of 192. Those early epochs are *easy* agreement with a nearly uninformative teacher.

The diagnostics say something is wrong from about epoch 50 (Table 4).

- **Effective rank of the CLS features peaks** at epoch 50 (centered, 51.1) and epoch 52 (uncentered, 48.2). It then falls by 63% and 52%, to 19.1 and 23.3 at epoch 99.
- **In the centered run,** `cls_std` **peaks** at epoch 61 and falls by about 40% by epoch 99. The uncentered run's peaks later, at epoch 79, and falls less.
- **Patch locality is best** at epoch 50 / 44 and then degrades by 27% / 18%.

The gap is about 50 epochs, and that is the point of the exercise.

- **The loss's biggest improvements happen during the decline.** The centered loss falls 74% ($0.250 \to 0.065$) between epochs 50 and 99, exactly while the representation loses more than half its effective dimensions.
- **Agreement gets easier as both sides get simpler.** The feature loss measures agreement with a target that is itself a function of the parameters, so a falling feature loss is consistent both with learning and with **dimensional collapse**. Only the diagnostics tell them apart.
- **Why the effective rank is the right signal here.** The collapse is *partial*: `cls_std` stays at 0.035–0.049 and never approaches 0, so this is not a constant representation. That is why the effective rank, which is scale-invariant and blind to a fully constant output, is the informative diagnostic here, with `cls_std` confirming the representation is not constant.
- **Limitation:** we did not save the epoch-50 checkpoints, so we cannot report the probe accuracy at the diagnostics' best epoch.

Q7. What does centering do? Is the collapsed solution still a minimiser, or is it just harder to find?

Probe accuracy of the uncentered run: 58.7% (mean pool) / 58.0% (CLS). The centered `feat75` gets 57.2% / 55.7%, so removing centering did *not* hurt. The uncentered run also ends with a *higher* `cls_std` (0.049 vs. 0.035) and effective rank (23.3 vs. 19.1).

What the code actually does. `methods.py` updates the center with the batch mean of the *already-centered* features:

$$\text{center} \leftarrow m\,\text{center} + (1-m)\,(\mu - \text{center}) \quad \Longrightarrow \quad \text{center}^\star = \mu/2.$$

So it subtracts only **half** the mean teacher feature. Textbook DINO uses $\text{center} \leftarrow m\,\text{center} + (1-m)\,\mu$, with fixed point μ. On the trained `feat75` teacher we measure $\|\text{center}\|/\|\mu\| = 0.5000$ and $\cos(\text{center}, \mu) = 1.0000$.

centering	distance from collapse ε	loss of constant student	mean pairwise cos of targets
starter code (center $\to \mu/2$)	0	0.0000	1.000
starter code (center $\to \mu/2$)	0.01	0.0003	1.000
DINO (center $\to \mu$)	0	0.1411	1.000
DINO (center $\to \mu$)	0.01	2.0008	0.009

Table 5: A constant (or nearly constant, scale ε) encoder and teacher, with a decoder that outputs c, after 150 centering updates.

Reading the evidence (Table 5).

1. **Existence: centering does not make the collapsed solution stop being a minimiser.**
   - **The starter code's centering:** a constant encoder gives the target normalize$(c - c/2) = c/\|c\|$, so the loss is exactly 0. That is proved above and checked numerically.
   - **Even correct DINO centering adds no penalty term.** The student's loss is still $2 - 2\cos$, which is minimised by matching *any* target, constant or not.
   - **At exact collapse the DINO target becomes $0/0$.** The surviving direction is whatever transient or rounding residual $c -$ center leaves (the nonzero DINO loss at $\varepsilon = 0$). A constant student can match that too, so a degenerate minimiser remains.
2. **Dynamics: correct centering makes collapse *repelling*.** Just next to collapse ($\varepsilon = 0.01$) the two versions behave completely differently.
   - **DINO centering** subtracts the shared component and ℓ_2-normalises what is left, which blows the tiny image-specific deviations up to unit length. The targets become diverse (mean cosine ≈ 0.01), and a constant student scores loss ≈ 2.0, about as bad as it gets. The gradient therefore pushes *away* from collapse.
   - **The starter's half-centering** leaves the targets identical (cosine ≈ 1.000, loss ≈ 0). Near collapse it neither repels nor attracts more than no centering.
   - **So centering's real effect is on attractiveness**, the optimiser's ability to find or stay at the collapsed point, not on existence. This is DINO's "centering acts across images": it is built from batch statistics of the *target*, not a term in the student's loss.
3. **What our runs support, carefully.**
   - **Our runs cannot show centering preventing collapse.** The uncentered run did not collapse in 100 epochs, and the two runs' diagnostic curves overlap until about epoch 50.
   - **Collapse was already non-attractive without centering.** That is consistent with the BYOL-style asymmetry that remains: stop-gradient on the teacher, a slow EMA ($m = 0.996$), a predictor (the decoder), and asymmetric inputs (the student sees 25% of the patches, the teacher sees all of them).
   - **Centering did not prevent the late dimensional decline either.** Both runs suffered it, and in principle it cannot: centering is a rank-one correction (subtract one mean vector) that leaves the rest of the spectrum alone.

- **Supported:** centering, in this code and in general, does not remove the collapsed minimiser. At best (correctly implemented) it makes the optimiser less likely to approach it.
- **Not supported by our data:** that centering is what kept `feat75` from collapsing.
- **A fair test** would use the textbook update, train long enough (or with weaker asymmetry) for the uncentered run to collapse, and use at least two seeds.

Q8. Required diagnostics for every run

Values are taken at the final epoch (99), on the fixed held-out batch logged by `train.py`, plus the effective-rank peak over training.

- **Effective rank** is scale-invariant, so it cannot see a constant representation.
- **Output std** can see a constant representation; ≈ 0 means complete collapse.
- **CLS–patch similarity** and **patch locality** describe the *patch* features, which the linear probe is blind to.

run	eff. rank (CLS)	eff. rank peak (@ epoch)	output std (CLS)	CLS–patch sim	patch locality	probe mean (%)	probe CLS (%)	final loss
`mae_r0.15`	10.2	15.1 @ 8	0.0074	0.083	0.158	57.74	50.29	0.1281
`mae_r0.50`	18.3	18.5 @ 90	0.0132	0.432	0.144	63.44	60.98	0.2338
`mae75`	17.7	17.8 @ 90	0.0313	0.170	0.121	68.19	60.68	0.3866
`mae_r0.90`	17.9	18.1 @ 89	0.0399	0.194	0.131	66.49	63.50	0.5736
`mae_block`	22.0	22.5 @ 90	0.0423	0.208	0.136	68.52	65.99	0.5565
`mae_grid`	18.8	18.8 @ 96	0.0175	0.333	0.142	65.19	64.76	0.3394
`feat75`	19.1	51.1 @ 50	0.0345	0.094	0.116	57.23	55.72	0.0651
`feat_nocenter`	23.3	48.2 @ 52	0.0488	0.180	0.109	58.65	58.04	0.0845

Table 6: Required diagnostics for all eight runs (final epoch unless stated).

3 Part 3: Connect

2. The target as a choice of equivalence

A target defines an equivalence. A target $Y = g(\text{patch})$ defines the relation $p \sim p' \iff g(p) = g(p')$, and the loss is **indifferent to every difference inside a class**. The lecture puts it this way: augmentation chooses what *input* variation to ignore, and the target chooses what *target* variation to ignore. A finer relation means the objective asks about more detail; a coarser one makes it more invariant.

Where each target sits:

- **MAE (pixels, as in our code).** With `norm_pix`, $g(p) = (p - \bar{p})/s_p$. Two patches are equivalent iff they differ by a per-patch brightness shift and contrast scale ($p' = ap + b$, $a > 0$). This is almost the identity, **the finest relation**. It treats every pixel-level difference beyond local brightness and contrast as worth predicting, and it is fixed by hand.
- **BEiT.** $g = \tau(p) \in \{1, \ldots, 8192\}$: a partition into 8192 cells, **fixed in advance** by an external tokenizer. Two slightly different browns can share a code, and then the objective

stops asking about that difference. It is coarser than pixels, but the classes cannot adapt as the representation improves.

- **iBOT / our feature target.** $g_\xi(p) = \text{normalize}\big(f_\xi(p \mid \text{whole image}) - \text{center}\big)$. The equivalence classes are the level sets of the teacher's feature directions. They are:
  - **contextual:** the same pixels in two images can be inequivalent, and different pixels equivalent;
  - **scale-free:** the normalisation discards scale;
  - **learned and moving:** they change as ξ is updated. This is the **coarsest and only adaptive** relation of the three.

On a scale: pixels (finest, fixed) $\prec$ BEiT tokens (8192 classes, fixed) $\prec$ iBOT features (learned, can coarsen without limit). Our MAE target sits at the fine end and our feature target at the coarse, adaptive end.

Our data show the adaptive relation coarsening during training. After epoch 50 the effective rank of the feature run fell from about 51 to 19: more and more images map to the same few directions. On this small model and 100-epoch schedule the fine target won the linear probe clearly (68.2% vs. 57.2%), the opposite of the large-scale literature. A coarse target only helps if its classes line up with semantics, and that needs a good teacher, which a small model in 100 epochs does not provide.

What a target that is *too* invariant looks like.

- **The extreme.** The trivial relation, where every patch is equivalent to every other: $g \equiv$ const. The target carries no information and the objective is solved by the constant solution of Q5, which **is collapse**.
- **The milder, practically relevant version.** A target invariant to everything except the image's global content, so that all patches *within* an image are equivalent. That rewards patch tokens that simply copy the image vector: CLS–patch similarity goes to 1 and patch locality is lost. This is the dense collapse DINOv3 runs into (next question).

3. Global versus dense structure

Existence vs. attractiveness, again. As with BYOL, the constant solution **exists** as a global minimiser of the feature objective: loss exactly 0, verified in Q5, and still true with this code's centering (Q7). What keeps training away from it is the **dynamics**, not the loss landscape. Our uncentered run did not collapse at all in 100 epochs.

How the anti-collapse mechanism differs from Mini-Project I.

- **Contrastive objectives** (negatives / uniformity) change **existence**. The repulsion term is in the student's loss, so the constant map is no longer a minimiser.
- **BYOL** keeps the constant minimiser and avoids it through **dynamics**: predictor, stop-gradient and EMA target.
- **The iBOT-style objective inherits BYOL's asymmetry**: EMA teacher, stop-gradient, decoder as predictor, plus a new input asymmetry (the student sees 25% of the patches, the teacher the whole image). It adds **centering**, which differs from both earlier mechanisms in two ways:

- **It acts on the target, through batch statistics, not as a term in the student's loss.** Near collapse it re-normalises the image-specific deviations to unit size (Q7), so it shapes *attractiveness* rather than existence.
- **It is defined at one level, the batch mean over all patches of all images.** It stops every patch of every image sharing one direction, but says **nothing about whether the patches within one image become alike**. Neither does the probe.

Why `patch_locality` can degrade while probe accuracy holds steady. Let $P \in \mathbb{R}^{N \times d}$ be the ℓ_2-normalised patch features of one image.

- **The probe sees only first moments.** It reads either the CLS vector or the mean $\frac{1}{N}\mathbf{1}^\top P$.
- **`patch_locality` sees second moments.** It is a function of the **Gram matrix** $G = PP^\top$ (which patches are most similar to which), mapped onto grid positions.
- **The two can change independently.** Many P share the same mean but have completely different G. We took `feat75`'s patch tokens for 256 test images and randomly permuted them within each image. The mean-pooled feature, and so the probe's input, is **unchanged to machine precision** (max difference 2.4×10^{-7}). Patch locality jumps from 0.12 to 0.38, which is chance level (0.375 for random positions on an 8×8 grid). The probe is *structurally* blind to the dense structure.
- **What DINOv3 measured.** Patch features drift toward the global vector, so CLS–patch similarity rises. That can leave classification untouched, or even help it by making the mean cleaner.

What our runs show:

- **Feature runs.** After epoch 50, patch locality degraded by 27% (centered) and 18% (uncentered) while the loss kept improving. CLS–patch similarity did *not* rise in these runs (centered $0.16 \to 0.09$), so the degradation is not DINOv3's CLS-drift mechanism. It goes together with the effective-rank decline: patches losing diversity rather than copying the CLS.
- **Pixel runs.** The DINOv3 signature does appear: CLS–patch similarity rose from $0.13 \to 0.43$ at ratio 0.50 and from $0.08 \to 0.33$ with grid masking, and those runs still probe at 63–65%.
- **Limitation:** we only have final-epoch probes, so we cannot show probe accuracy flat over the same epochs as the locality decline. Across the eight runs, final locality and probe accuracy are only weakly related (correlation +0.09).

The link to Gram anchoring. DINOv3 adds $\mathcal{L}_{\text{Gram}} = \|G_{\text{student}} - G_{\text{anchor}}\|_F^2$, using an earlier checkpoint whose dense features were still good.

- **It regularises exactly what our diagnostic watches and the probe ignores.** G is the object `patch_locality` is computed from.
- **It does not freeze the features.** G is invariant to rotations of the feature space: for any orthogonal R, $(PR)(PR)^\top = PP^\top$. So anchoring preserves the *pairwise relations between patches* while leaving the global representation free to keep improving.
- **For our runs,** the natural anchor is the feature run's epoch-50 checkpoint, where patch locality and effective rank are at their best. Gram anchoring from there targets precisely the decline in Figure 4.

- **As the lecture stresses, it is a repair applied late, not a removal of the cause.** The diagnostics tell you when it is needed; the loss and the probe do not.

AI-use acknowledgement

[Describe how generative AI was used, as required by the syllabus. For example: running the experiments, drafting the analysis, and code for figures and tables. Also state that the group reviewed and is responsible for the interpretation.]